Homework 4: Lightfields and Focal Stacks

Andrew Krapivin

Oct 30, 2026

1 Lightfield Rendering, Depth from Focus, and Confocal Stereo

1.1 Initials

Pixel (y, x) of the 6400×11200 lenslet image lies under lenslet $(s, t) = (\lfloor y/16 \rfloor, \lfloor x/16 \rfloor)$ at position $(u, v) = (y \bmod 16, x \bmod 16)$. So I reshape the image to $(400, 16, 700, 16, 3)$ and transpose the axes to $(1, 3, 0, 2, 4)$, giving $L(u, v, s, t, c)$ of shape $16 \times 16 \times 400 \times 700 \times 3$.

1.2 Sub-aperture views

The sub-aperture view at (u, v) is the 400×700 image `L[u, v]`. To build the mosaic, `tile_grid` transposes the 16×16 grid of views to $(16, 400, 16, 700, 3)$ and reshapes it to $6400 \times 11200 \times 3$, so view (u, v) is the tile in row u and column v (Fig. 1). This is the inverse of the rearrangement above, applied with the roles of (u, v) and (s, t) swapped. Neighboring views are basically the same, differing only by a small parallax.

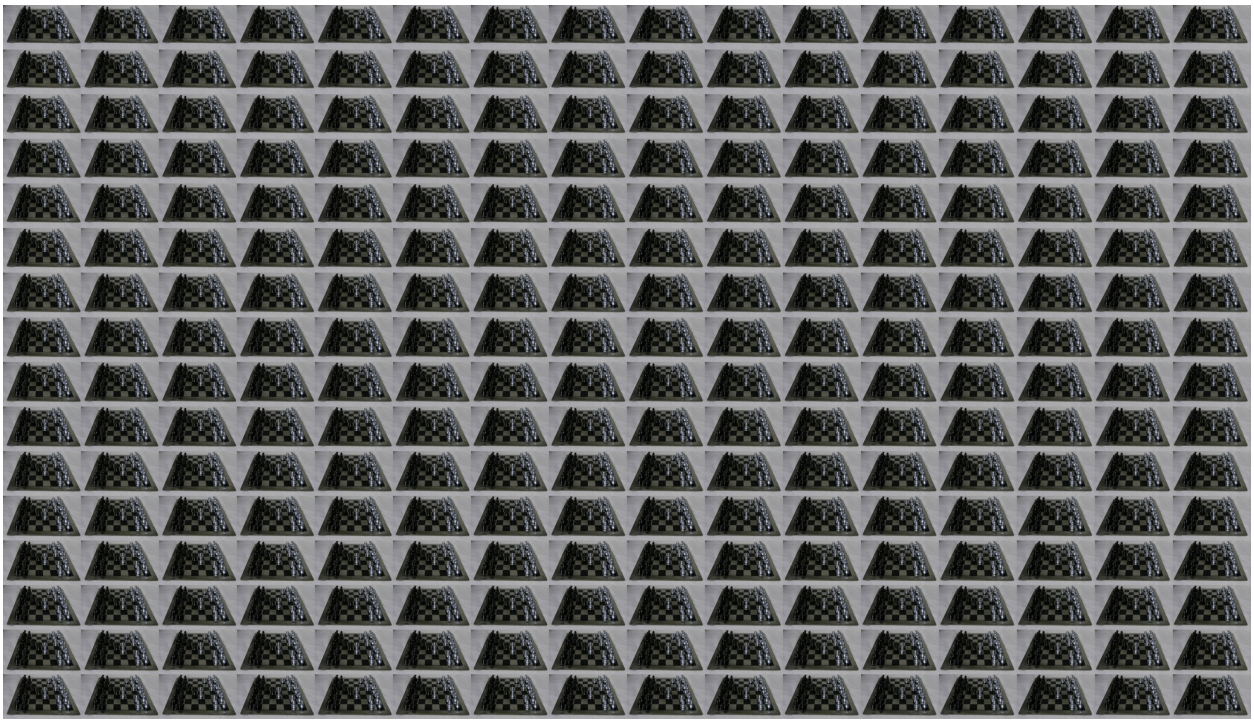


Figure 1: Mosaic of the 16×16 sub-aperture views; u increases downward and v to the right.

1.3 Refocusing and focal-stack simulation

For each (u, v) in the aperture, measured from the lenslet center, I shift the sub-aperture view so that pixel (s, t) samples $L(u, v, s + d\,u, t - d\,v)$, then average the shifted views (dividing by $|A|$ keeps the exposure of a single view). On the image, increasing s moves down and increasing t moves to the right. The t shift has the opposite sign from Eq. (2) because in this lightfield the views move down (increasing s) as u increases but left (decreasing t) as v increases. The fractional shifts use bilinear interpolation (manually implemented for performance), and samples outside the view take the nearest border pixel. The focal stack uses a square aperture of side 8 (the central 8×8 views) and depths from $d = -2$ to 2 with a 0.1 sized increment. This range covers the whole scene: the back of the board is in focus at about $d = -0.3$ and the front at about $d = 1.6$.

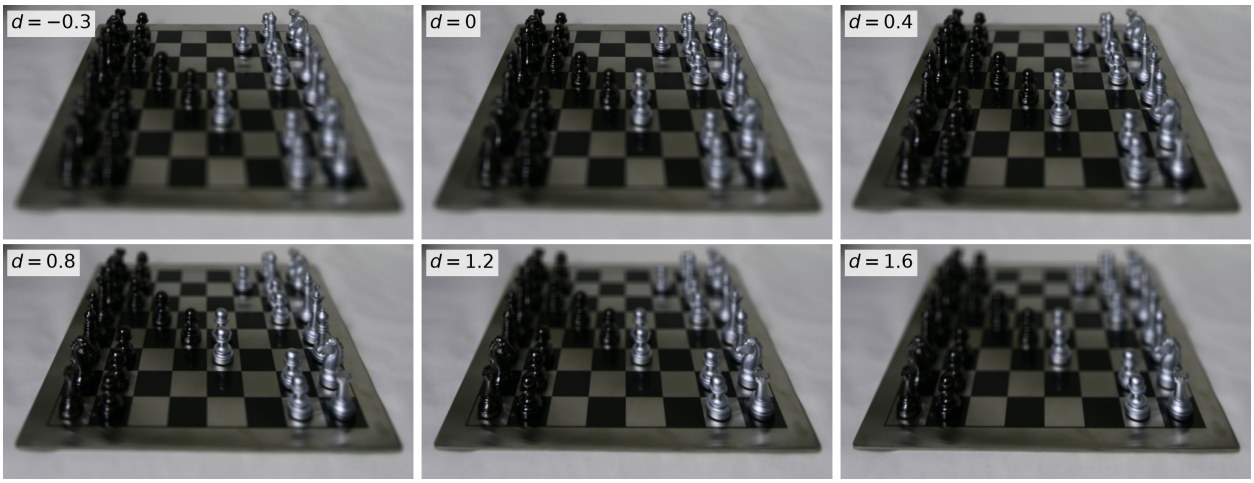


Figure 2: Refocusing with the central 8×8 views at six depths, from the back of the board ($d = -0.3$) to the front ($d = 1.6$). All 41 depths of the focal stack are in `data/part1/refocus/`.

1.4 All-in-focus image and depth from focus

Sharpness is computed on the luminance of the linearized (inverse-gamma) stack. Both Gaussian blurs are applied to each slice separately (`gaussian_filter` with $\sigma = (0, \sigma, \sigma)$), so they never mix depths, and the resulting weights are shared by all three color channels. The all-in-focus image and the depth map are then the w-weighted averages of the stack and of d, with a small ϵ in the denominator.

I tried $\sigma_1 \in \{0.5, 1, 2, 4, 8\}$ with $\sigma_2 = 4$ and $\sigma_2 \in \{1, 2, 4, 8, 16\}$ with $\sigma_1 = 2$. Increasing σ_1 makes the all-in-focus image blurrier, while changing σ_2 does not affect it much. The depth map changes with both. A large σ_1 counts mid frequencies as sharp, even though they survive defocus, so the cloth becomes speckled and the halos thicken. A small σ_2 makes flat regions noisy, and a large one blurs the pieces into blobs. $\sigma_1 = 2$ and $\sigma_2 = 4$ work best, and I use them from here on.

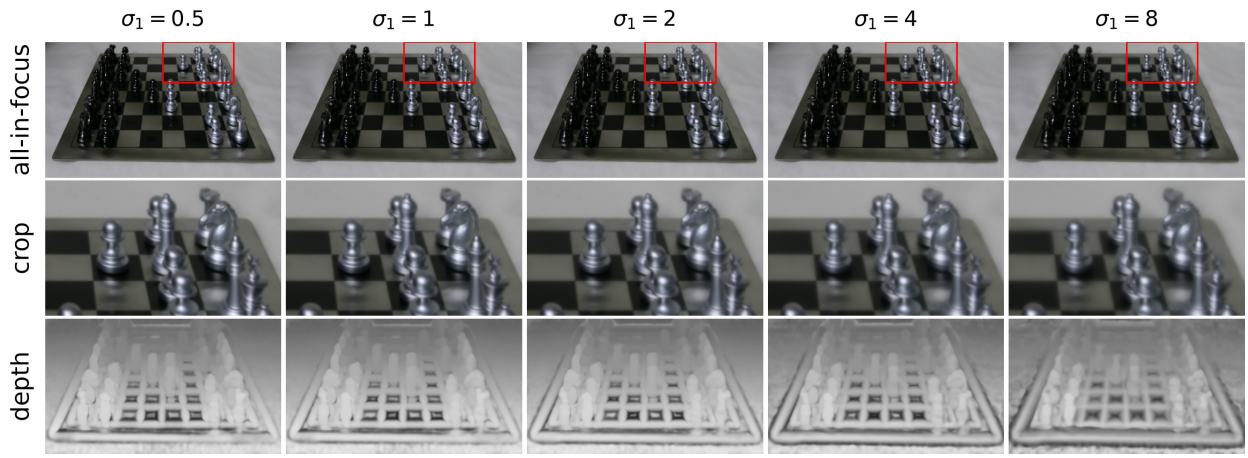


Figure 3: Varying σ_1 with $\sigma_2 = 4$: all-in-focus image (top), its red box enlarged (middle), and depth map (bottom). Depth maps show d from -2 (black) to 2 (white), so brighter is closer to the camera.

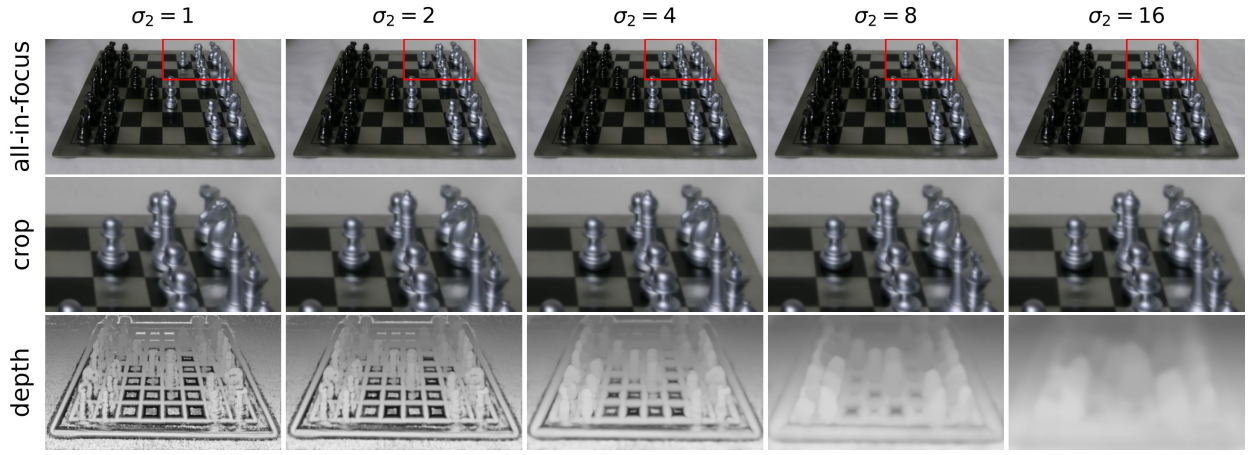


Figure 4: Varying σ_2 with $\sigma_1 = 2$, laid out as in Figure 3.

The depth is wrong where there is no texture: inside the plain checker squares, on the white cloth, and in halos around the pieces and the board's edge. Inside a uniform square, the only high frequencies come from nearby edges once they are defocused enough to blur into the square. So the weights peak at the wrong depth, and the squares appear too far away. σ_2 also spreads each edge's sharpness to its neighbors, which causes the halos.

These depth errors barely show in the all-in-focus image. A textureless region looks the same in every slice of the stack, so even weights that peak at the wrong depth give the right color there. The errors are only visible in the halos, where some weight goes to defocused copies of the edges and the piece outlines become slightly soft.

1.5 Focal-aperture stack and confocal stereo

I use square apertures of side $2, 4, \ldots, 16$, each refocused at the same 41 depths. Eq. (7) of Hasinoff and Kutulakos divides each aperture's intensity by its exitance E_{xy}. As the handout suggests, I set $E_{xy} = |A|$, the number of (u, v) pairs in the aperture, and `refocus` already divides by $|A|$, so

no extra normalization is needed. At the in-focus depth, every aperture sees the same point, so the AFI column is constant. Following their Eq. (8), I take the depth of each pixel to be the d whose column has the smallest variance across apertures, summed over color channels. The full stack (8×41 refocused images) is stored as float32, about 1.1 GB, and the variance is accumulated one channel at a time so the stack is never copied. In Fig. 6, this gives $d = 1.0$ for A and $d = 0$ for B.

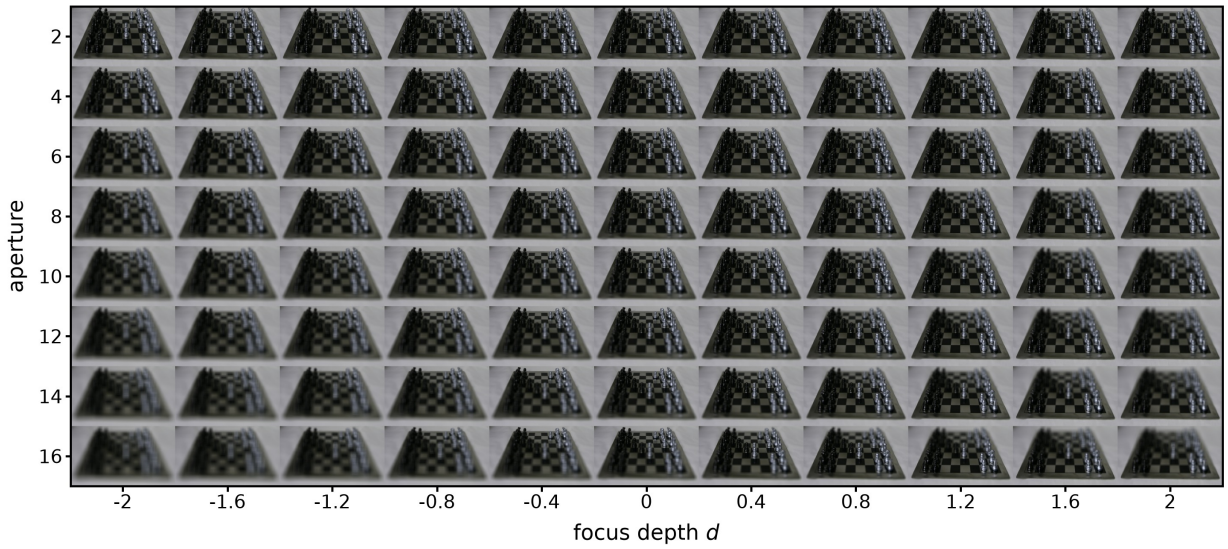


Figure 5: Collage of the focal-aperture stack: square apertures of side 2 to 16 (rows) and every fourth of the 41 focus depths (columns).

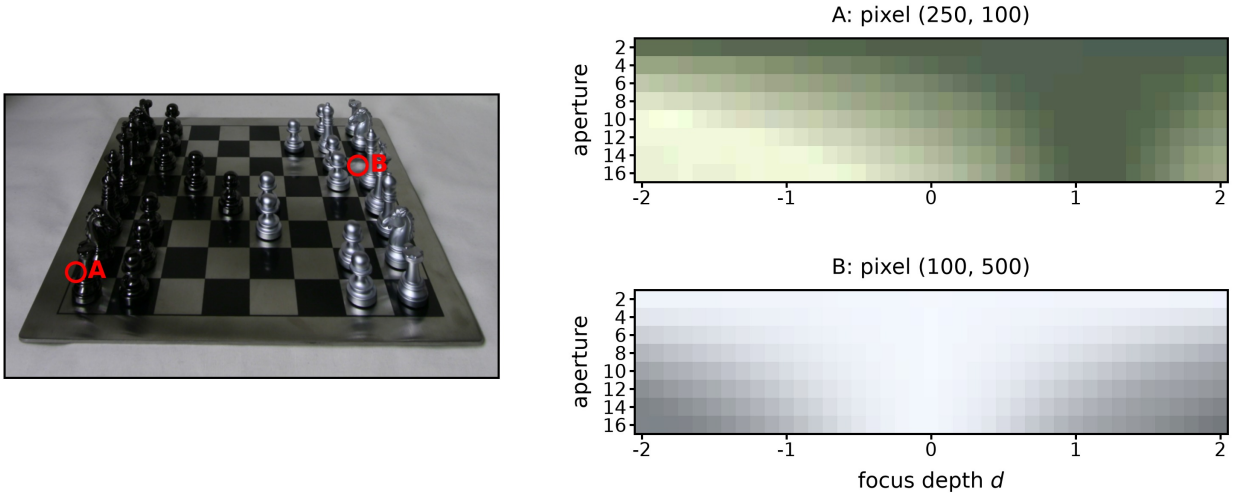


Figure 6: AFIs of the two pixels marked on the central view. Rows are apertures and columns focus depths; each AFI is scaled to its own maximum.

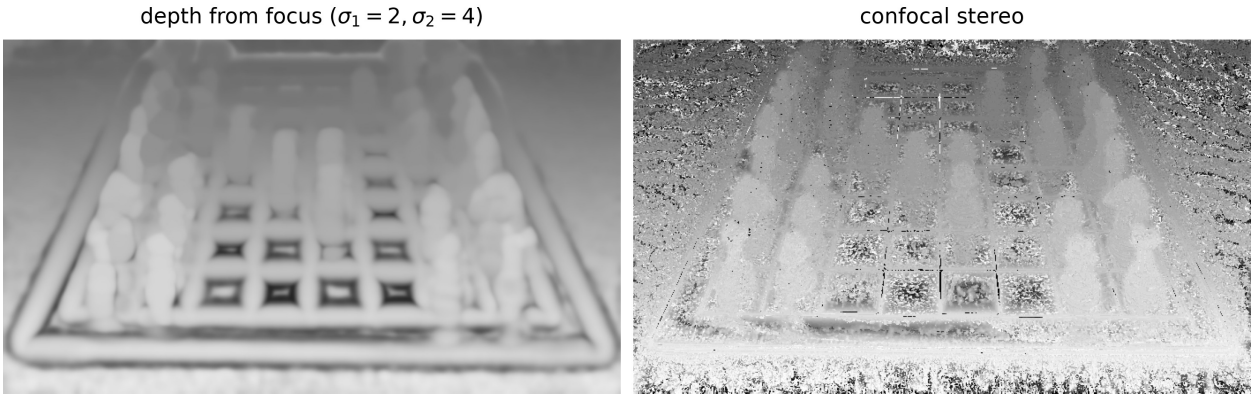


Figure 7: Depth from focus ($\sigma_1 = 2$, $\sigma_2 = 4$) and confocal stereo, on the same scale (d from -2 in black to 2 in white).

On textured pixels, the two methods agree to within about one depth step. Confocal stereo works per pixel, so it has sharp outlines, no halos, and follows the board's slope where depth from focus does not. But it is very noisy wherever the AFI barely changes with focus, such as on the cloth and inside the squares. Depth from focus is smoother because σ_2 pools over a neighborhood, and that pooling also causes its halos.

3 Capture and Refocus Your Own Lightfield

3.1 Capturing an unstructured lightfield

I captured three videos with my phone (an iphone 15 pro using ProCam to fix the exposure, ISO, white balance, focus, and shutter speed; apologies that I did not record the settings, every video was filmed at different times with different settings), each of a scene with objects at clearly different depths (Table 1). I first recorded a video at CMU of *Hamerschlag* Hall. I was moving my phone freely in a zig zag, trying to keep it as steady as possible but without anything to remove rotations. Then I took two videos at home, pressing my phone to my (flat) window to capture the scene outside; here I was hoping to get a more planar video. I recorded two versions: one with bars in the front and one at the top of my window where there are no bars. I was curious if the processing technique could completely blur out the bars, but this is not turn out to work. The videos are in `data/part3/` (`chipotle_bars.mov` re-encoded to fit GitHub's 100 MB limit).

Video	Capture	Motion	Frames used	Processed at
`chipotle.mov`	3840×2160, 30 fps, 21 s	pressed to window	every 2nd (320)	960×540
`chipotle_bars.mov`	3840×2160, 24 fps, 47 s	pressed to window	every 3rd (375)	960×540
Hamerschlag	1920×1080, 30 fps, 29 s	hand-held	every 2nd (428)	960×540

Table 1: The captured videos. All frames are resized to 960×540 before processing.

3.2 Refocusing an unstructured lightfield

Normalized cross-correlation. Matching runs on the luminance (Y of XYZ) of the linearized frames. For an $m \times m$ template g ($n = m^2$ pixels), let $g_0 = g - \bar{g}$ and let B be an $m \times m$ box filter of value $1/n$, so $\bar{I}_t = I_t \star B$. Since g_0 has zero mean, the numerator of Eq. (9) is just $I_t \star g_0$, and

the window variance is $I_t^2 \star B - \bar{I}_t^2$, so

$$h_t = \frac{I_t \star g_0}{\sqrt{\sum g_0^2 \cdot n \, \max\big(I_t^2 \star B - \bar{I}_t^2,\ 0\big) + \epsilon}}$$

needs one correlation (`scipy.signal.correlate2d`, `valid` mode) and two separable box filters. This matches OpenCV's `TM_CCOEFF_NORMED` to within 1.3×10^{-7}.

Shifting and averaging. I pick a 60×60 template in the middle frame and search for it in every frame inside a square window centered on it. The shift is the template's position minus the argmax of h_t (Eq. (10)). Frames are shifted by integer cropping, and each pixel of the sum is divided by the number of frames that cover it, so the borders do not darken. A whole-frame search window let 37–53 of the 375 frames match the wrong place, creating ghosts, so I size the window to the template plus the largest true shift: 140 pixels for the chipotle videos (shifts of about $\pm$10–30) and 260 for Hamerschlag (about $\pm$90).



Figure 8: `chipotle.mov`: templates (red boxes) on the middle frame, and the refocused images, from the spire (far) to the brick pillar (near). The parked car disappears from every result because it drove away during the capture.

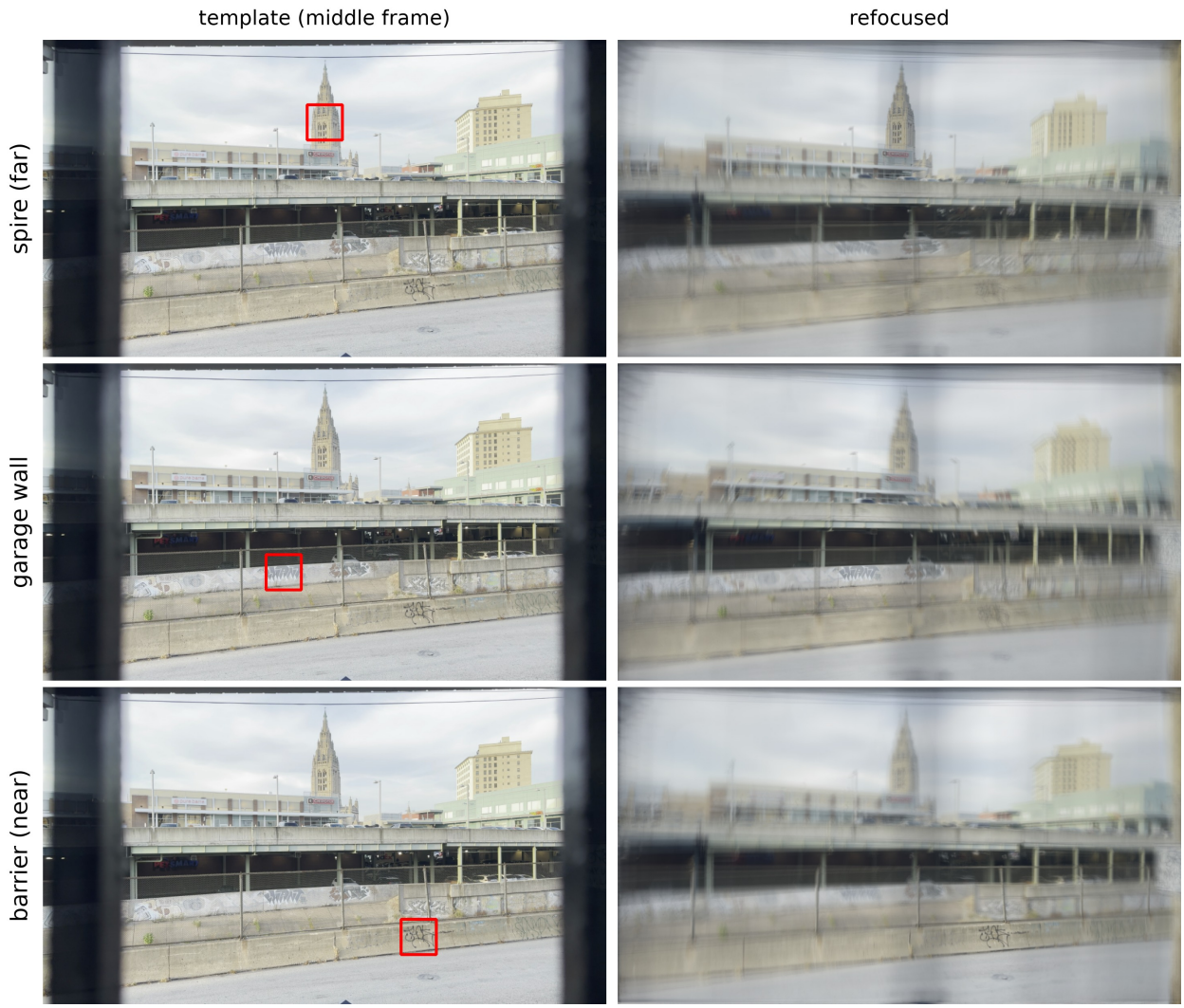


Figure 9: `chipotle_bars.mov`. The dark window bars near the camera sweep across the scene and leave a faint vertical band in the results.

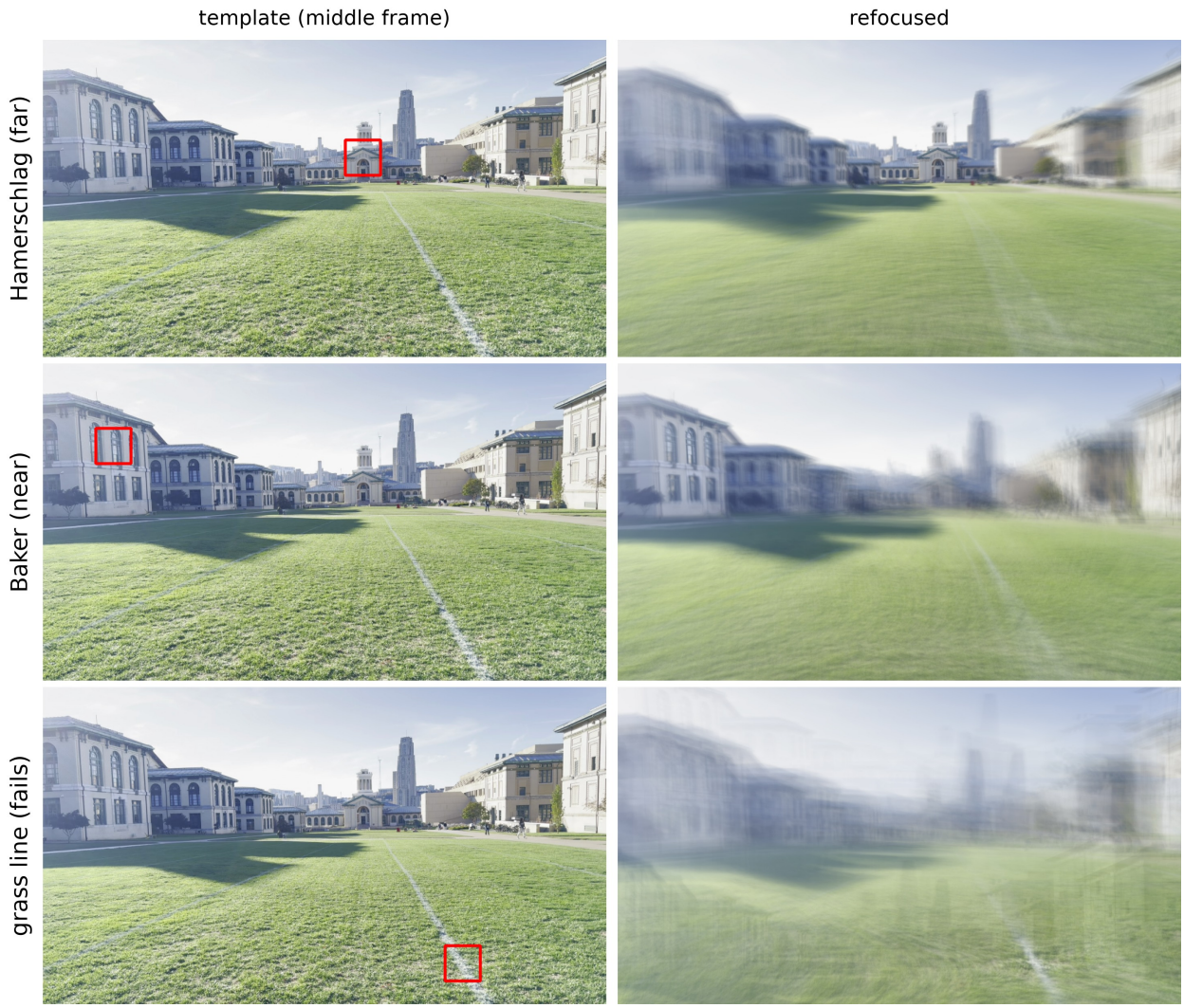


Figure 10: Hamerschlag (hand-held). Focusing on Hamerschlag Hall keeps it and the Cathedral of Learning sharp and blurs the buildings on either side. Focusing on Baker Hall (left) blurs the background. The grass-line template fails (see text).

Failures. Template matching fails when the template does not look the same in every frame:

- **Moving objects.** The parked car in `chipotle.mov` drove away during the capture, so it cannot be matched and vanishes from the averages.
- **Occlusion.** The window bars in `chipotle_bars.mov` cover the templates in about 31–35 of the 375 frames; those frames match the wrong place and slightly blur the in-focus object.
- **The ground plane.** The grass-line template in the Hamerschlag video loses the match in about 20% of the frames, which ghosts the buildings (Fig. 10). Even with perfect matches, only a thin strip of ground could be in focus, since it spans a range of depths.

3.3 Camera rotation and the shape of the defocus

Shift-and-add assumes the camera only translates. Small pans and tilts shift the whole image, so template matching absorbs them, but roll about the optical axis rotates each aligned frame about the template and smears everything else along arcs (Fig. 11). Tracking two distant points in `chipotle.mov` shows up to 3.2° of roll, which moves a point 300 pixels from the template by about 16 pixels, comparable to the real parallax. Pressing the phone to the glass prevents pan and tilt but not roll, which is why, although it refocuses better than the hand-held video, the resulting image still appears smeared.

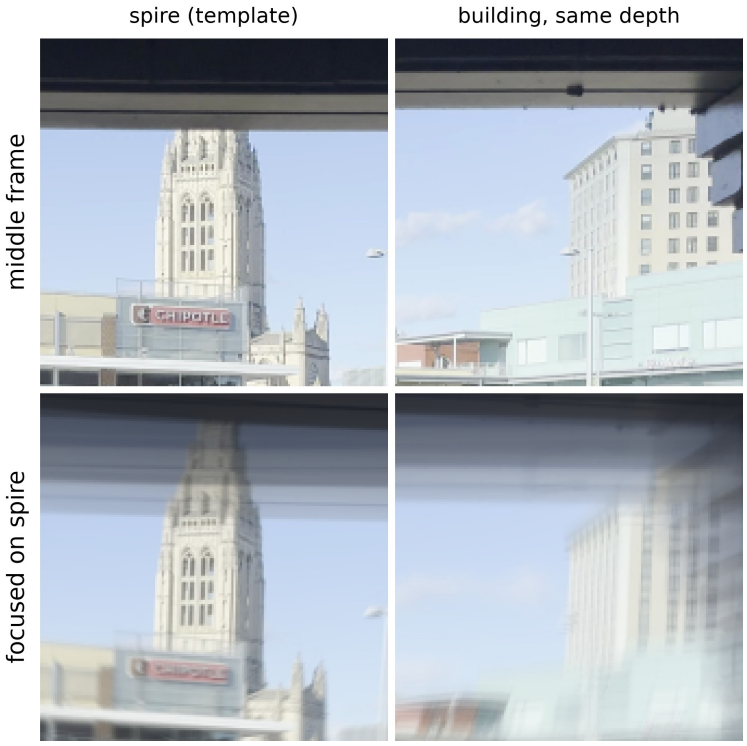


Figure 11: Crops of `chipotle.mov` focused on the spire. The spire (the template) is sharp. A building at about the same distance, 270 pixels to the right, is smeared vertically, because the camera roll rotates each aligned frame about the template.

4 Bonus: Capture and Process Your Own Focal-Aperture Stack

4.1 Capture

I captured a table of different (edible and decorative) squashes in front of a paper screen with a Nikon D3400 and a 35 mm lens, mounted on a tripod. The camera is controlled over USB with gphoto2 (`src/take_focal_stack.py`), so it is not touched during the capture, at a fixed ISO 400 in NEF and JPEG. For each of four apertures (f/1.8, f/4, f/9 and f/22), the script scales the shutter speed by N^2 to keep t/N^2 (mostly) constant (1/125 s to 1.3 s), drives the focus to the end of its range so every sweep starts from the same setting, and takes 30 shots with a fixed focus step between them. Each sweep runs in one `gphoto2 --shell` session, since manual focus driving only works in live view. Focus setting 0 is the farthest (the screen) and focus moves toward the camera with the index (Fig. 12); I use settings 5 to 22, where something in the scene is in focus.

4.2 Radiometric calibration and alignment

`src/focal_aperture.py` reads the NEFs linearly with `rawpy`, resizes them to 800 pixels wide, and divides each by its t/N^2 from the EXIF, which puts every aperture on the same radiometric scale and plays the role of the exitance normalization in Hasinoff and Kutulakos's Eq. (7). Refocusing also changes the magnification, so I estimate a scale and translation per focus setting with ECC (`cv2.findTransformECC`) on the always-sharp f/22 sweep (the scale grows from 0.995 to 1.018, about 10 pixels at the edges), apply it to every aperture, and crop the undefined border, giving 782×516 images.



Figure 12: Example frames at the widest and narrowest apertures, after calibration and alignment. Focus moves from the screen (setting 5) to the front of the table (setting 22).

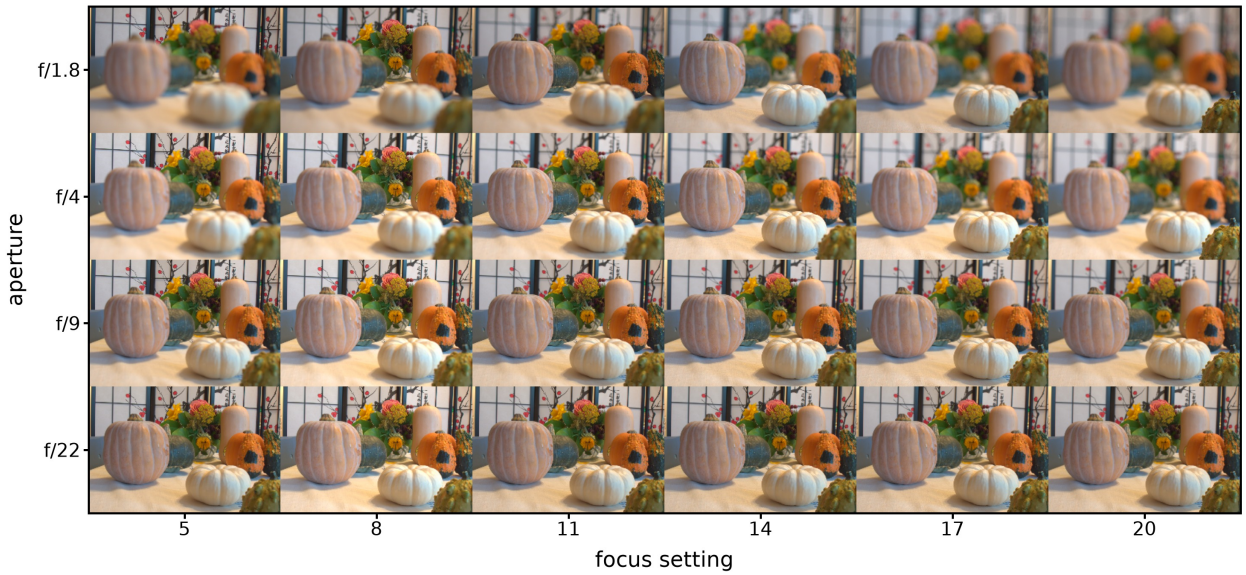


Figure 13: Collage of the captured focal-aperture stack: every third focus setting from 5 to 22.

4.3 All-in-focus image and depth maps

Depth from focus and the all-in-focus image use the same code as Part 1 (Eqs. (3)–(8), $\sigma_1 = 2$, $\sigma_2 = 4$) on the f/1.8 sweep. Confocal stereo uses the same Eqs. (7) and (8) of Hasinoff and Kutulakos as Part 1, on all four apertures. Depth is in units of focus setting.

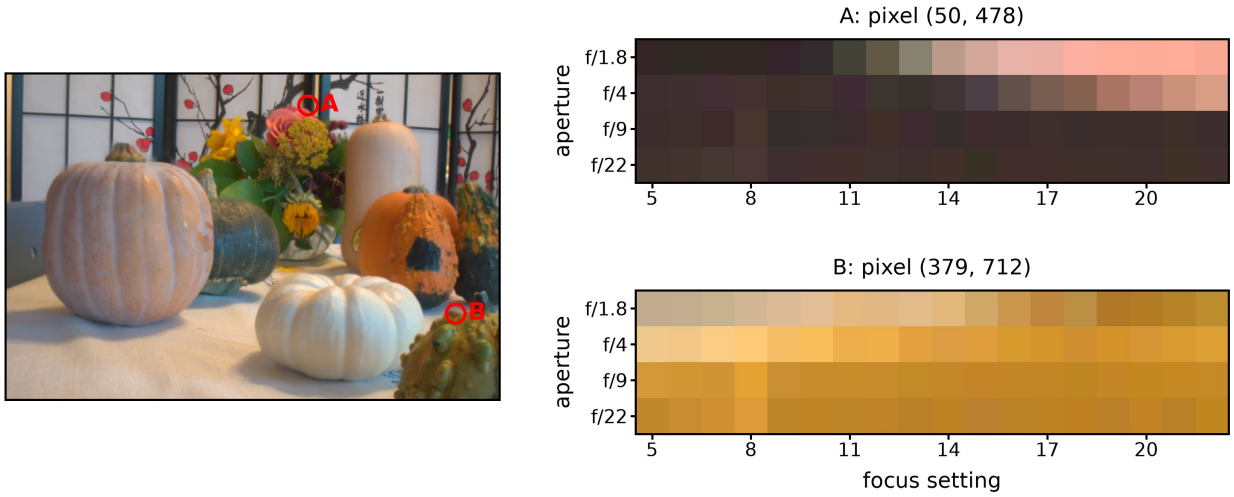


Figure 14: AFIs of a branch on the screen (A) and the bumpy gourd at the front (B). Each AFI is scaled to its own maximum.

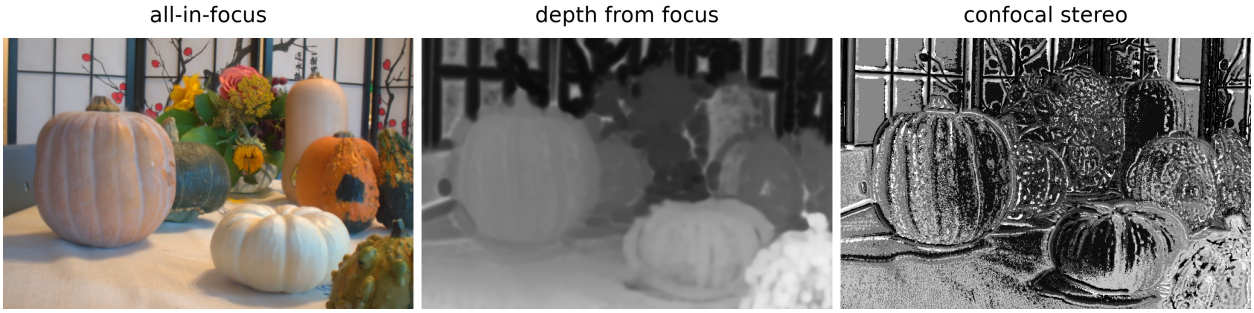


Figure 15: All-in-focus image, and depth from focus and confocal stereo depth from focus setting 5 (black) to 22 (white), so brighter is closer.

The depth from focus map is reasonably accurate: the little squash at the front shows up as the closest image, followed by the white pumpkin, followed by the koginut, and so on. However, some of the background in the japanese room divider at the back appears close, as the screen has a very uniform texture. By contrast, the confocal stereo map is somewhat all over the place: it in some areas captures depth correctly, but the screen in the back is shown as very close, and half of the white pumpkin shown as far, for example.